

- 1

저장소 링크: <https://github.com/huggingface/cadgenbench/tree/main>

▼ To-do List

1. Python, GitHub, CAD library, LLM API + build123d/CadQuery 관련 개념
2. 어떤 LLM을 사용하고, 어떤 입력값을 받는가
3. LLM이 생성한 CAD code가 어떤 방식으로 실행이 되는지
4. build123d/CadQuery 등 어떤 CAD backend를 사용하는지
5. 최종적으로 STEP 파일이 어느 경로에서 생성되는지
6. 생성 오류가 났을 때 agent가 어떻게 수정·반복하는지

→ 우선적인 목표는 "사용자의 자연어 요청 → CAD 생성 → STEP 파일 출력"이 실제로 동작하는 최소 예제를 재현

→ 이걸 완료 후, 우리 목적에 맞게 입력 형식, 지원 부품 종류, 검증 방식 등을 수정·확장

⇒ 개인과제 -1의 우선적인 목표: 설치부터 실행까지 진행해보고, 실행 결과와 전체 pipeline 구조를 간단히 정리해서 공유

▼ 전체적인 구조

텍스트 → *CAD*코드 → *3D*모델 → *STEP*

▼ 최종 목표

: 결국 LLM으로 CAD에서 형상을 만들어주는 코드를 받고 결과물 (STEP을 받으면?), 우리가 간단하게 CATIA로 만든 파일과 비교해서 정확도를 테스트?

<전체 흐름>

```
부품 요구사항 또는 공학 도면
↓
LLM이 CAD 생성 코드 작성
↓
build123d/CadQuery로 코드 실행
↓
AI가 만든 output.step
↓
정답 CAD 모델과 비교
↓
정확도와 실패 원인 분석
```

여기서 LLM은 STEP 파일의 내용을 직접 작성하기보다, 보통 다음과 같은 Python CAD 코드를 작성합니다.

CATIA 파일과 비교한다는 의미

우리가 CATIA로 정확한 부품을 직접 만든다면 그것을 **정답 모델, 즉 Ground Truth**로 사용할 수 있습니다.

단, CATIA의 원본 **.CATPart** 파일을 그대로 비교하기보다 CATIA에서 다음처럼 STEP으로 내보내는 것이 좋습니다.

```
CATIA로 직접 만든 정답 모델
↓ STEP으로 Export
reference.step

LLM Agent가 만든 모델
↓
output.step
```

그 후 두 STEP 파일을 비교합니다.

```
reference.step ↔ output.step
```

즉, 비교 대상은 다음과 같습니다.

- `reference.step` : 사람이 CATIA로 정확하게 만든 정답
- `output.step` : LLM Agent가 생성한 결과

▼ 최종 목표를 고려해 무엇을 해야하는가?

1. 파일 유효성

AI가 만든 STEP 파일이 정상적으로 열리는지 확인합니다.

- CATIA에서 열리는가?
- 하나의 정상적인 Solid인가?
- 면이 깨지거나 비어 있지 않은가?
- STEP 파일은 생성됐지만 형상은 없는 상태가 아닌가?

2. 주요 치수

정답과 AI 결과의 치수를 비교합니다.

- 전체 길이
- 너비
- 높이
- 지름
- 구멍 지름
- 구멍 위치
- 벽 두께
- 필렛 반지름

예:

측정 항목	CATIA 정답	AI 결과	오차
전체 높이	30 mm	30 mm	0 mm
외경	40 mm	40 mm	0 mm
구멍 지름	10 mm	8 mm	2 mm

3. 전체 형상

전체 부피나 표면이 얼마나 일치하는지 비교합니다.

- 부피 차이
- 표면 간 거리
- 두 모델이 겹치는 비율

- 외곽 크기(Bounding Box)

4. 형상의 구조

구멍과 솔리드의 개수가 맞는지 확인합니다.

예:

정답: 하나의 몸체 + 관통 구멍 2개
AI 결과: 하나의 몸체 + 관통 구멍 1개

겉모양이 비슷해도 구멍 개수가 다르면 잘못된 모델입니다.

5. 설계 특징

- 관통 구멍인가, 막힌 구멍인가?
- 필렛이 들어갔는가?
- 모따기가 있는가?
- 구멍이 정확한 면에 있는가?
- 대칭 구조가 맞는가?

▼ 전체 학습 순서

⇒ CADGenBench에는 STEP 결과를 평가하는 코드가 구현

1. CADGenBench 제공 샘플로 baseline이 실제 작동하는지 재현
2. CATIA로 간단한 정답 부품 제작
3. 같은 부품을 자연어로 Agent에 요청
4. AI 결과 `output.step` 생성
5. CATIA 정답 `reference.step` 과 비교
6. 치수·형상·구멍·성공 여부 기록
7. 자연어 표현이나 부품 복잡도를 바꿔 반복 실험

+) 예를 들어 첫 실험 부품은 다음 정도가 적당합니다.

가로 50mm, 세로 30mm, 높이 10mm인 직육면체 판을 만든다.
중앙에 지름 10mm의 관통 구멍을 만든다.
모서리 가공은 하지 않는다.
단위는 mm이다.

이 부품을 CATIA로 직접 만들어 `reference.step` 으로 저장하고, 같은 설명을 Agent에 넣어 `output.step` 을 받는 것입니다.

▼ baseline 실행을 통해 교수님이 요청사항 파악

- 어떤 입력을 LLM에 전달하는지
 - 어떤 LLM을 선택하는지
 - 어떤 prompt를 사용하는지
 - LLM이 어떤 CAD 코드를 만드는지
 - 코드가 어떻게 실행되는지
 - build123d가 어떻게 STEP을 만드는지
 - 오류가 나면 어떻게 수정하는지
 - 결과 파일이 어디에 저장되는지
-

▼ STEP 파일 의미

- STEP 파일: **3차원 CAD 모델을 저장하고 교환하는 표준 파일 형식**
- CAD 생성 파이프라인:

사용자 요구사항

↓

LLM이 요구사항 해석

↓

CAD Python 코드 작성

↓

build123d/CadQuery가 코드 실행

↓

3D 형상 및 STEP 파일 생성

↓

오류·형상 검사

↓

문제가 있으면 LLM이 코드 수정

↓

최종 STEP 파일

▼ Python, GitHub, CAD library, LLM API 관련 개념

▼ build123d

: Python으로 기계 부품과 3D 형상을 만드는 CAD 라이브러리로 CADGenBench의 기본 backend가 build123d

<주요기능>

- 박스, 원통, 구 등의 기본 형상 생성
- 형상 합치기
- 형상 빼기
- 모서리 둥글게 만들기
- 스케치 돌출
- 구멍 생성
- STEP 파일 내보내기

▼ OpenCascade

: **정밀한 3D CAD 형상을 계산하는 핵심 엔진**, 즉 CAD 커널

<CAD 커널>

→ CAD 프로그램의 가장 안쪽에서 실제 수학 계산을 수행하는 엔진

⇒ 복잡한 계산을 OpenCascade가 처리

<build123d와 OpenCascade의 차이>

: build123d는 사람이 Python으로 CAD를 쉽게 작성할 수 있게 해주는 상위 라이브러리입니다. OpenCascade는 그 아래에서 실제 형상 계산

사람 또는 LLM

↓

build123d Python 코드(형상 제작을 위해)

↓

OpenCascade

↓

정밀한 3D 형상

⇒ 사람이 보는 build123d 코드

```
outer = Cylinder(20, 20)
hole = Cylinder(5, 20)
part = outer - hole
```

내부에서는 OpenCascade가 원통을 만들고 두 형상의 차집합을 계산

▼ CadQuery

:Python 기반의 CAD 라이브러리입니다. build123d와 목적은 비슷하지만 코드 작성 방식과 일부 기능이 다릅니다

<예시 코드>

```
import cadquery as cq

part = (
    cq.Workplane("XY")
    .circle(20)
    .extrude(20)
    .faces(">Z")
    .workplane()
    .hole(10)
)

cq.exporters.export(part, "output.step")
```

→ 처음에는 기본값인 `build123d` 만 사용하는 것이 좋습니다. 두 라이브러리를 동시에 배우면 혼동될 수 있기 때문입니다

▼ BREP 파일

:build123d와 CadQuery 내부의 OpenCascade 계열의 CAD 엔진, Boundary Representation의 약자로, 3차원 물체를 면과 모서리의 관계를 포함한 정밀 형상 파일

<예>

3D 부품

├─ 면(Face)
├─ 모서리(Edge)
└─ 꼭짓점(Vertex)

→ 이런 정밀한 형상 표현 덕분에 단순한 이미지나 삼각형 메시보다 치수 기반 설계에 적합

▼ CADbackend

: 실제로 CAD 형상을 계산하고 파일을 만드는 내부 엔진 또는 라이브러리

LLM이 CAD 코드 생성

↓

CAD backend가 코드 실행

↓

3D 형상과 STEP 파일 생성

⇒ CADGenBench에서 선택할 수 있는 backend는 대표적으로 다음 두 개입니다.

- build123d
- CadQuery

즉, LM은 설계 아이디어와 코드를 만들지만 실제 3차원 형상을 계산하는 것은 CAD backend입니다.

▼ STEP과 STL의 차이

항목	STEP	STL
표현 방식	면과 곡면 기반 CAD 형상	삼각형 메시
치수 정확성	높음	메시 해상도에 영향받음
CAD 수정	비교적 적합	어려움
주요 용도	설계 교환·제조	3D 프린팅
확장자	<code>.step</code> , <code>.stp</code>	<code>.stl</code>

이번 과제에서는 설계 결과를 평가하고 다시 편집해야 하므로 STEP이 중요

▼ STL

: 3차원 형상을 삼각형 mesh로 저장하는 파일입니다. 주로 3D 프린팅에 사용됩니다.

▼ API

: 한 프로그램이 다른 프로그램의 기능을 요청하는 인터페이스
보통 ChatGPT 웹사이트에서는 사람이 직접 메시지를 입력 하지만, API를 사용하면 Python 프로그램이 자동으로 LLM에 메시지를 보내고 응답을 받을 수 있습니다.

<예>

CADGenBench Python 프로그램

↓ API 요청

OpenAI·Anthropic·Google 서버

↓ API 응답

생성된 CAD 코드

▼ Status Code — 확장

: API 요청 결과를 나타내는 숫자입니다.

- 200 : 성공
- 400 : 잘못된 요청
- 401 : 인증 실패
- 429 : 호출 한도 초과
- 500 : 서버 오류

▼ Local Repository

: 내 컴퓨터에 있는 Git 저장소입니다

▼ Remote Repository

: GitHub에 있는 원격 저장소입니다.

▼ Clone

: 원격 저장소 전체를 내 컴퓨터로 복제

▼ Rendering / 렌더링

: 3D 데이터를 사람이 볼 수 있는 2D 이미지로 만드는 과정

▼ PyVista

: Python에서 3D 형상과 mesh를 시각화하는 라이브러리

▼ CADGenBench

: AI가 기계 CAD 부품을 얼마나 정확하게 생성하거나 편집하는지 평가

▼ LiteLLM

: OpenAI, Anthropic, Google은 API 사용법이 조금씩 다릅니다. LiteLLM은 이 차이를 통일해주는 중간 라이브러리입니다.

CADGenBench

↓

LiteLLM

├── OpenAI API

├── Anthropic API

└── Gemini API

▼ LLM API

: Python 프로그램이 GPT 같은 LLM에게 자동으로 질문하고 답을 받는 통신 방법

<구조>

사용자의 자연어

↓

LLM API

↓

LLM이 CAD Python 코드 작성

↓

CADGenBench가 코드 실행을 관리

↓

build123d

↓

OpenCascade가 실제 3D 형상 계산

↓

STEP 파일

+) Python 프로그램 → 인터넷 → LLM 서버 → 응답

<왜 API가 필요한가?>

CAD Agent가 자동으로 반복하려면 사람이 매번 ChatGPT에 복사해서 붙여넣을 수 없습니다.

API를 이용하면 다음 과정이 자동화됩니다.

1. 요구사항 전송
2. CAD 코드 응답 수신
3. 코드 실행
4. 오류 발생
5. 오류 메시지를 LLM에 다시 전송
6. 수정된 코드 수신
7. 재실행

⇒ 즉:

- LLM API: 모델과 통신하는 방법
- API 키: 모델 사용 권한
- LiteLLM: 여러 LLM API를 비슷한 방식으로 호출하는 도구

▼ CADGenBench

: CADGenBench 자체가 GPT 같은 인공지능 모델은 아닙니다. 여러 구성요소를 연결하고 실행하는 Python 프로젝트

<전체역할 3가지>

1. CAD 생성 문제 제공

부품 설명, 공학 도면, 기존 STEP 파일 같은 입력 샘플을 제공합니다.

2. Baseline Agent 제공

LLM을 호출하고 CAD 코드를 실행하는 기본 Agent가 구현되어 있습니다.

입력
→ LLM 호출
→ 코드 생성
→ 실행
→ STEP 생성
→ 렌더링
→ 오류·형상 피드백
→ 코드 수정

우리는 이 baseline을 처음부터 만들 필요 없이 먼저 실행해볼 수 있습니다.

3. 결과 평가

생성한 STEP이 정답 형상과 얼마나 비슷한지 평가하는 구조를 제공합니다.

대표 평가 내용은 다음과 같습니다.

- STEP 파일이 유효한가?
- 형상이 정답과 비슷한가?
- 관통 구멍과 부품 연결 구조가 맞는가?
- 중요한 결합 부위가 올바른가?

<두 종류의 작업>

- Generation: 설명이나 도면으로 새 CAD 모델 생성
- Editing: 기존 STEP 파일을 요청에 맞게 수정

<CADGenBench 통상적인 구조>

CADGenBench

- LLM에게 코드 생성 요청
- 생성된 코드를 실행
- build123d가 OpenCascade 사용
- STEP 파일 생성

개념	정체	이번 과제에서 하는 일
LLM API	프로그램과 LLM의 통신 방법	자연어를 보내고 CAD 코드를 받음
CADGenBench	CAD Agent 실행·평가 프로젝트	LLM 호출, 코드 실행, 수정 반복, 결과 관리
OpenCascade	정밀 CAD 계산 엔진	실제 3D 형상 계산과 STEP 생성

⇒ LLM API로 CAD 코드를 받아오고, CADGenBench가 전체 과정을 관리하며, OpenCascade가 실제 3D 형상을 계산

▼ CAD 생성의 기본 원리

: Python 코드

↓

build123d

↓

OpenCascade

↓

STEP 파일

[Baseline 첫 번째 시도 ⇒ 실패]

▼ baseline을 실행하기 전에 준비 작업

- 저장소 clone — 완료
- Python 3.12 준비 — 완료
- 가상환경 생성 — 완료
- 필요한 라이브러리 설치 — 진행 중(CADGenBench가 요구하는 라이브러리 목록을 `pyproject.toml` 에서 확인 후 설치 단계)
- 공개 샘플 데이터 설정
- 사용할 LLM과 API 키 준비
- 샘플 하나로 baseline 실행
- `output.step` 확인

▼ baseline 두가지 명령 (run만 사용)

- `run`: 실제 LLM Agent를 실행해 STEP 파일 생성
- `package`: 생성 결과를 리더보드 제출용 ZIP으로 묶기

샘플 입력
→ baseline run
→ LLM 호출
→ CAD 코드 생성·실행
→ output.step

▼ 첫 baseline 실행

이제 첫 baseline을 실제로 실행할 차례입니다. 이 명령부터는 OpenAI API를 호출하므로 **소액의 API 비용이 발생할 수 있습니다.**

첫 실행은 저장소 예시에 나온 샘플 `101`, 기본 backend인 `build123d`, 최대 3회 반복으로 제한하겠습니다.

```
bat
cadgenbench baseline run 101 --model openai/gpt-5.5 --backend build123d --max-iter 3 --max-tokens 20000
```

설정 의미:

- `101`: 첫 번째 실험 샘플
- `openai/gpt-5.5`: OpenAI 모델 사용
- `build123d`: CAD Python 코드 backend
- `--max-iter 3`: 최대 3회 생성·수정
- `--max-tokens 20000`: 전체 token 상한
- `--max-tokens-per-call 6000`: 한 번 호출의 출력 상한
- `--max-duration 600`: 최대 10분
- `--reasoning-effort low`: 추론 비용을 낮게 설정

첫 baseline 재현에 성공했습니다. 종료 표시는 `done: no` 였지만, **유효한 STEP 파일은 실제로 생성됐습니다.**



결과:

- `output.step` 생성 성공
- 유효한 CAD 형상
- Watertight 형상
- Solid 1개
- Face 71개
- 체적: `693,854.52`
- 전체 크기: `204 x 200 x 45`
- 위상 오류 없음

<실행 과정>

- GPT-5.5가 첫 번째 build123d 코드 생성
- 코드 실행 성공
- STEP 파일 생성
- 형상을 이미지로 자동 렌더링
- 렌더 결과를 보고 GPT-5.5가 두 번째 수정 코드 생성
- 두 번째 코드는 `Locations()` 에 잘못된 정수 인자를 전달해 실패
- 총 token이 25,424 로 설정한 20,000 을 넘어서 추가 수정 없이 종료
- 마지막으로 성공했던 첫 번째 STEP을 최종 결과로 보존

<두번째 코드의 오류>

```
ValueError: Locations doesn't accept type <class 'int'>
```

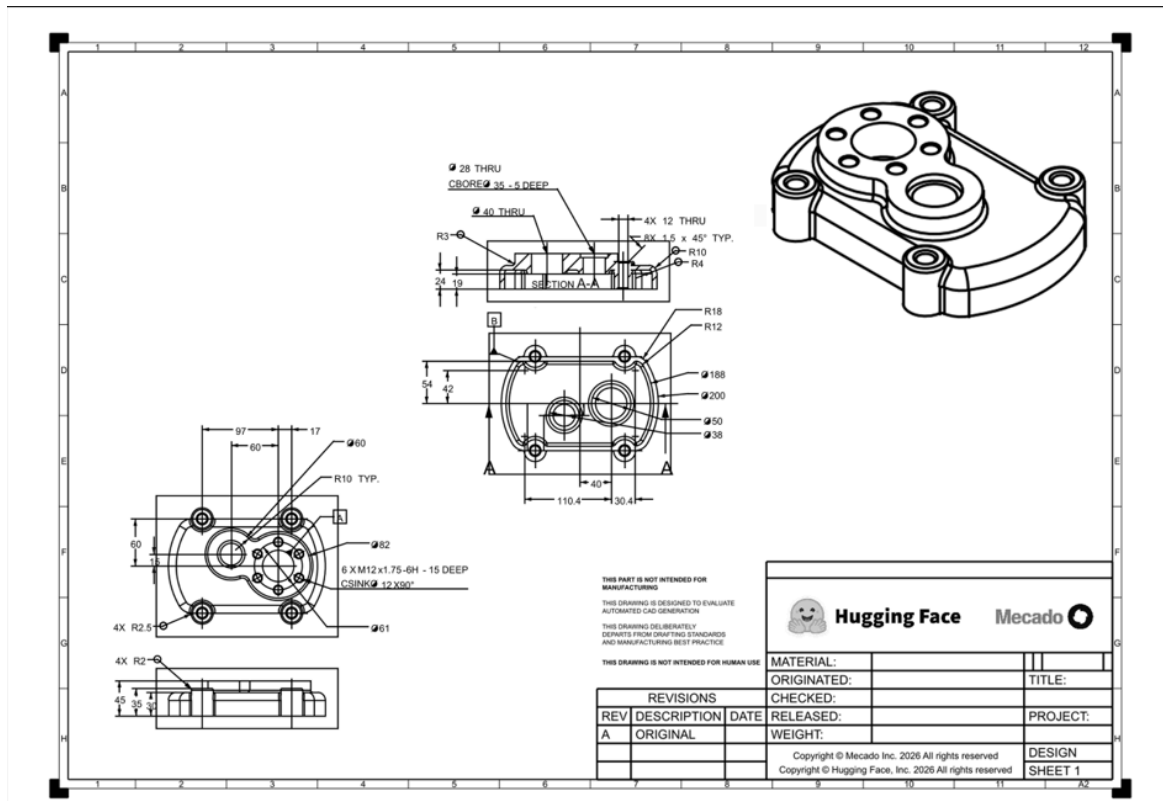
즉, Agent의 자동 수정 과정까지 실제로 관찰했습니다. 다만 수정 코드가 오히려 실패했고 token 예산이 부족해 다시 고치지는 못했습니다.

이번 실행에서 확인한 문제점:

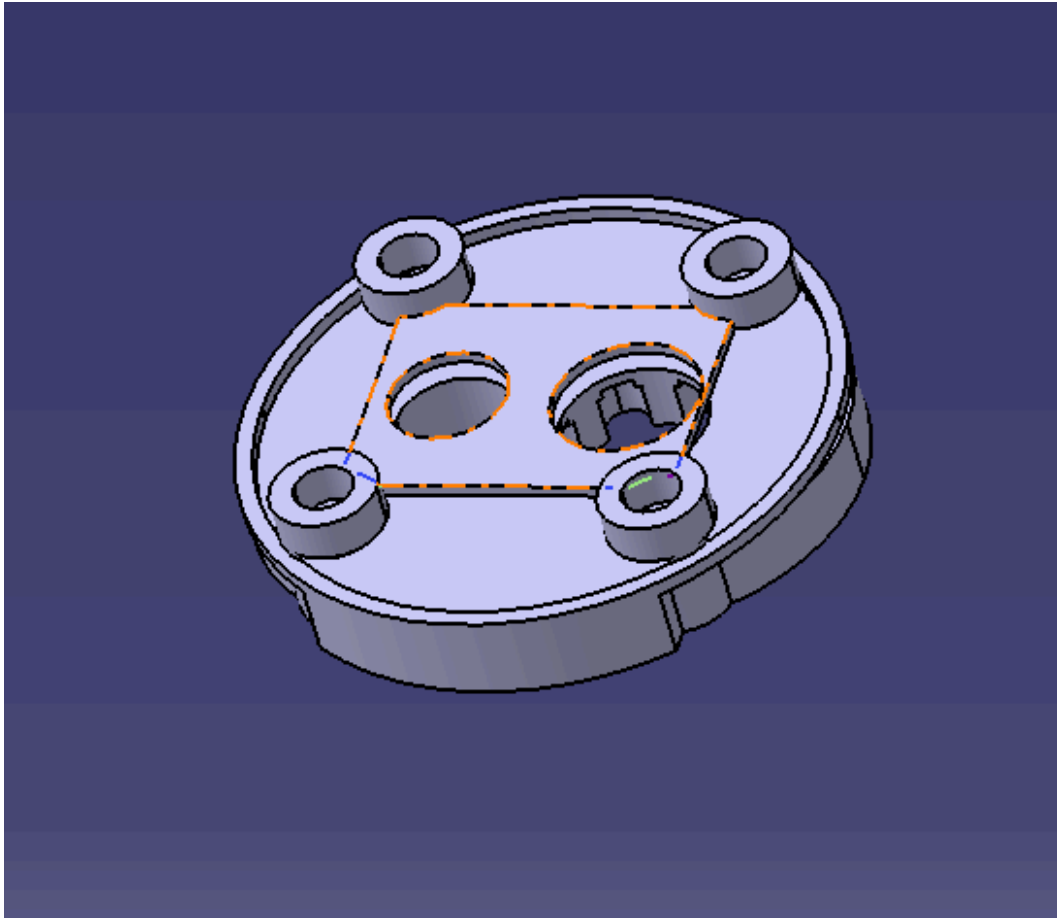
- LLM의 첫 코드가 성공해도 후속 수정이 실패할 수 있음
- build123d API 사용법을 LLM이 잘못 생성
- `-max-tokens 20000` 인데 한 호출이 추가되면서 실제 사용량이 25424 까지 넘어감
- 최종 STEP은 유효하지만 도면과 기하학적으로 정확한지는 아직 공식 평가하지 않음
- 공식 ground truth가 비공개이므로 현재는 유효성만 로컬에서 확인됨

▼ 첫 baseline 결과값

- input 도면



- LLM을 통해 제작한 output.step 파일 (CATIA 도면)



결론적으로 입력값과 차이가 있음.

이번 결과의 판정

평가 항목	결과
STEP 파일 생성	성공
CATIA에서 열기	성공
정상 Solid	성공
전체 높이	대체로 일치 가능
전체 외곽 형상	크게 불일치
구멍 및 보스 구성	일부만 구현
도면 재현 정확도	낮음

추가로 보이는 차이

화면상 다음 부분도 도면과 다릅니다.

- 중앙의 6개 체결 구멍이 없음

- 상부의 두 원형 돌출부 형상이 지나치게 단순함
- 네 모서리 보스의 연결 형상이 다름
- 외곽의 직선 구간과 둥근 모서리 구성이 사라짐
- 보스 사이의 선·리브가 도면과 다른 위치에 생성됨
- 카운터보어와 단차 구조가 정확한지 의심됨

지금은 세부 치수를 전부 쉘 필요가 없습니다. **전체 윤곽이 틀린 것은 가장 큰 1차 오류**

왜 자동 수정되지 않았나?

첫 코드가 원형 본체를 만들었고 STEP 생성에는 성공했습니다. Agent가 렌더링을 본 뒤 수정 코드를 작성했지만 다음 build123d 오류로 실행에 실패했습니다.

```
ValueError: Locations doesn't accept type <class 'int'>
```

그다음 token 한도에 도달해 세 번째 수정 기회를 갖지 못했습니다. 따라서 최종 파일에는 첫 번째로 성공한 잘못된 원형 모델이 남았습니다.

이번 사례는 교수님이 요청한 “어떤 문제가 있는지 식별”에 딱 맞는 결과입니다.

Agent는 파일의 유효성과 도면의 정확성을 구분하지 못했다. 첫 결과는 유효한 Solid 였지만 외곽 형상이 크게 틀렸고, 시각 피드백 후 생성한 수정 코드도 실행 오류가 발생해 개선되지 않았다.

다음에는 token 한도를 조금 늘려 Agent가 이 오류를 스스로 수정할 기회를 더 줄 수 있습니다.

▼ 첫 번째 실험을 통해 알게된 사실

▼ 어떤 LLM을 사용하고, 어떤 입력값을 받는가

- 첫 번째 실험에서 사용한 모델은:

```
OpenAI GPT-5.5
```

실행할 때 LiteLLM 형식으로 다음처럼 지정.

```
openai/gpt-5.5
```

CADGenBench가 OpenAI API를 직접 전용 코드로 호출하는 것이 아니라, 중간 라이브러리인 **LiteLLM**을 통해 호출합니다.

```
CADGenBench  
→ LiteLLM  
→ OpenAI API  
→ GPT-5.5
```

LiteLLM을 사용하기 때문에 모델 문자열과 API 키만 바꾸면 Gemini나 Claude도 사용

따라서 두번째 실험부터는 무료인 gemini 버전을 사용할 생각.

- 어떤 입력값을 받았는가?

⇒ 도면의 정보를 하나하나 입력하는 줄 알았는데, 그게 아닌 입력값을 도면으로부터 형상을 가능한 정확하게 재현하는 코드였다.

이러한 도면을 통해 LLM에 전달되는 정보들

- 공학 도면 이미지 `input.png`
- 부품을 재현하라는 작업 설명
- build123d 사용 지침
- build123d API 참고자료
- 반드시 STEP 파일을 생성하라는 출력 조건
- 이전에 생성한 코드
- 코드 실행 결과
- 오류 메시지

- 생성 형상의 렌더링 이미지

따라서 이번 baseline은 정확히 말하면:

```
순수 자연어 → CAD
```

보다는:

```
공학 도면 이미지 + 작업 지시문 → CAD 코드 → STEP
```

에 가깝습니다.

▼ LLM은 무엇을 출력했는가?

LLM의 출력은 **build123d**를 사용하라는 Python 코드.

```
공학 도면  
→ GPT-5.5  
→ build123d Python 코드
```

생성된 코드는 실행 결과 폴더에 저장됐습니다.

첫 번째 생성 코드:

```
turn_0/code_0.py
```

두 번째 수정 코드:

```
turn_1/code_0.py
```

따라서 우리가 원한다면 각 시도에서 LLM이 어떤 설계 판단을 했는지 코드를 직접 분석

▼ LLM이 생성한 CAD 코드는 어떻게 실행되는가?

LLM이 Python 코드 생성

↓

CADGenBench가 코드를 파일로 저장

↓

별도 Python 프로세스에서 코드 실행

↓

build123d가 CAD 형상 생성

↓

내부적으로 OpenCascade/OCP가 형상 계산

↓

output.step 내보내기

- GPT-5.5: CAD Python 코드 작성
- CADGenBench: 코드 저장·실행·검증·반복 관리
- build123d: Python CAD 모델링 인터페이스
- OpenCascade/OCP: 실제 정밀 형상 계산
- STEP: 최종 CAD 결과 형식

▼ 어떤 CAD backend를 사용하는가?

첫 번째 실험에서는 다음 backend를 사용했습니다.

```
build123d
```

명령어에 직접 지정했습니다

+) 그럼 왜 **CadQuery** 를 설치하지 않고 안 쓰는가?

⇒

1. 일단 **build123d** 가 기본 backend 모델
2. 설계 방식에서 차이가 발생

build123d

형상을 객체로 만들고 조합하는 방식이 직관적입니다.

일반적인 Python 문법과 유사해서 LLM이 각각의 형상을 변수로 나누어 작성하기 편합니다.

CadQuery

기존 작업면을 정하고 그 위에서 연속적으로 설계하는 방식

실제 CAD 프로그램의 스케치·피처 작업 흐름과 비슷합니다. 다만 중간에 면 선택이 잘못되면 이후 작업도 틀어질 수 있습니다.

+) 어떨 때 필요한가?

⇒ CadQuery는 build123d로 만들 수 없는 STEP 파일을 만들기 위해 필요한 것이 아니라, 작성 방식이나 LLM의 코드 생성 성공률이 더 유리한지 확인할 때 선택하는 대안 backend

▼ STEP 파일은 어디에 생성됐는가?

첫 번째 실험에서 최종 STEP 파일은 다음 경로에 생성

```
C:\Users\user\Documents\Codex\cadgenbench\results\20260715_205913_gpt-5.5\101\output.step
```

일반적인 저장 규칙은 다음과 같습니다.

▼ 생성 오류가 났을 때 agent가 어떻게 수정·반복하는지

Agent의 반복 구조는 다음과 같습니다.

1. LLM이 코드 생성
2. Python 코드 실행
3. STEP 파일 생성 여부 확인
4. STEP 형상 유효성 검사
5. 형상을 이미지로 렌더링
6. 실행 결과와 렌더 이미지를 LLM에 전달
7. LLM이 수정 코드 생성
8. 다시 실행

반복 종료 조건

- LLM이 완료했다고 판단
- 유효한 최종 결과 확정
- 최대 반복 횟수 도달
- 최대 token 도달
- 최대 실행시간 도달
- 복구할 수 없는 API 오류 발생

이번 실험에서는 토큰 수가 부족해서 실제로 자동 수정 과정이 발생

Turn 0

첫 번째 LLM 호출:

총 11,274 token
입력 9,589
출력 1,685

결과:

코드 실행 성공
실행시간 약 86초
STEP 생성 성공
자동 렌더링 성공

Turn 1

Agent가 첫 형상의 렌더링을 LLM에게 다시 보여준 뒤 수정 코드를 받았습니다.

```
총 14,150 token  
입력 12,441  
출력 1,709
```

하지만 수정 코드에서 다음 오류가 발생했습니다.

```
ValueError:  
Locations doesn't accept type <class 'int'>
```

문제가 된 형태:

```
Locations(left_center[0], left_center[1], base_thickness)
```

LLM이 build123d의 `Locations()` 사용법을 잘못 작성한 것입니다.

▼ 왜 `Locations()` 사용법을 잘못 작성하였는가?

LLM은 build123d 사용법을 정확히 조회해서 복사한 게 아니라, 학습한 패턴을 바탕으로 코드를 생성합니다. 그래서 `Locations()`의 인자 형식을 다른 CAD 함수와 혼동하거나 추측해서 작성할 수 있습니다.

⇒ 그럼 agent 설정을 더 세부적으로 가야한다고 생각. 왜냐면 무식하게 토큰 수를 늘려 수정을 반복할 순 없다고 생각.

Turn 2

Agent가 오류를 다시 수정하려 했지만 전체 token 사용량이 제한을 넘었습니다.

```
실제 누적 token: 25,424  
설정된 한도: 20,000
```

따라서 세 번째 LLM 수정 호출 없이 종료되었습니다.

```
stop: max_tokens
```

done: no

최종적으로 마지막 성공 결과였던 Turn 0의 STEP 파일이 보존됐습니다.

▼ 전체적인 pipeline

```
Hugging Face에서 샘플 101 다운로드
↓
input.png 공학 도면 로드
↓
CADGenBench가 prompt 구성
- 작업 설명
- 도면 이미지
- build123d 지칭
↓
LiteLLM을 통해 GPT-5.5 API 호출
↓
GPT-5.5가 build123d를 통해 Python 코드 생성
↓
CADGenBench가 별도 프로세스로 Python 코드 실행
↓
build123d/OpenCascade가 CAD 형상 계산
↓
output.step 생성
↓
STEP 유효성 검사
↓
PyVista/VTK로 이미지 렌더링
↓
렌더 이미지와 실행 결과를 agent가 GPT-5.5에 피드백
↓
수정 코드 생성
↓
수정 코드 실행 실패
↓
token 한도 초과로 종료
↓
마지막 성공 STEP 보존
```

결론: LLM이 build123d API를 잘못 사용한 것이 직접적인 오류 원인이며, 에이전트가 토큰 제한 때문에 오류를 수정하지 못한 것이 최종 실패의 추가 원인

▼ LLM의 비결정성

⇒

LLM이 코드를 데이터베이스에서 그대로 꺼내는 게 아니라, 다음에 올 단어(토큰)의 확률을 계산하며 하나씩 선택해서 생성하기 때문에, 같은 요청이어도 매번 생성 코드가 조금 달라질 수 있어서, 첫 실행에는 맞고 다음 실행에는 틀릴 수도 있습니다.

교수님 질문 사항

⇒ 그렇다면

1. 먼저 LLM의 비결정성을 고려를 해서 temperature 를 0으로 해서 동일한 상태로 한번 더 돌려봐야 하는것인가? 또는 기존 설정을 유지하고 agent의 프롬프트를 바꿔서 실행해야 하는 것인가?
2. 해당 github가 자연어 기반 cad 제작이 아닌 도면을 바탕으로 cad를 제작하는데 계속 진행해도 괜찮은 것인가? ⇒ ㄱ
3. 해당 CADGenBench가 이미지만 받는게 아니라 두 파트로 나뉨
 - `description.yaml` 의 자연어 설명
 - `input.png` 같은 첨부 이미지

현재 자연어 파트는 도면을 보고 형상을 최대한 정확하게 재현하라. 는 명명만 있어서 정확하지 않았던거 같은데, 그렇다면 해당 github의 목적과 다르게 자연어에 더 포커스를 맞춰서 진행을 해야하는 것인가?

4. 1번을 해보는 것이 좋은지 아니면
 - 새로운 실험: 이미지 + 상세 치수 명세
 - 추가 실험: 상세 치수 명세만 사용

이렇게 해보는 것이 좋은지.

[gemini API를 사용한 두 번째 실험]

실패

: Gemini는 오류 메시지를 받아 코드를 두 차례 수정했지만, 매번 다른 build123d 실행 오류가 발생했고 토큰 한도까지 사용한 뒤 STEP 파일 생성에 실패

[gemini API를 사용한 세 번째 실험]

변경 조건:

- 반복 횟수: 3 → 6
- 전체 토큰: 20,000 → 50,000
- 코드 실행 제한: 120초 → 240초
- 전체 실행시간: 600초 → 1,200초

실패 → 이번 결과는 **build123d API 환각과 에이전트의 반복 수정 실패**